

Q-Tab: Quantized Tabular Data Generator

Julian Wustl^{*1} Philipp Haid^{*1} Yarema Okhrin¹² Claudius Schnörr³

Abstract

Codebook-based generators built on masked language model (MLM) transformers have become highly effective in text and vision, yet remain underused for tabular data. This is because codebooks typically act as information bottlenecks, whereas synthetic tabular generation requires a code space larger than the training sample, with additional codes trained to support new tabular rows. We address this gap with Q-Tab, a codebook-based tabular generator that uses lookup-free quantization (LFQ) with residual corruption to jointly tokenize numerical variables, categorical variables are directly one-hot tokenized. A BERT-style MLM captures dependencies in the token space and can then be sampled from. Corruption propagates reconstruction supervision across the numerical code space, but under joint encoder-decoder training induces a moving-target regression problem whose difficulty depends on the corruption structure. This motivates residual LFQ as the quantization mechanism, balancing broader supervision with locality. Q-Tab achieves state-of-the-art predictive utility and label prediction, while matching the distributional fidelity of diffusion-based generators.

1. Introduction

Synthetic tabular data is increasingly important for privacy-preserving data sharing, simulation, and augmentation (Fonseca & Bacao, 2023; Hernandez et al., 2022; Assefa et al., 2021). Although generation is, in principle, just sampling from a multivariate distribution, tabular data combines heterogeneous feature types, multimodal behaviour, and often

^{*}Equal contribution ¹Department of Statistics and Data Science, University of Augsburg, Germany ²Department of Economics and Statistics, Linnaeus University, Sweden ³Department of Computer Science and Mathematics, Munich University of Applied Sciences, Germany. Correspondence to: Julian Wustl <julian1.wustl@uni-a.de>, Philipp Haid <philipp1.haid@uni-a.de>.

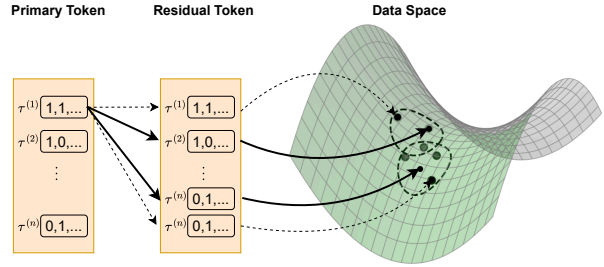


Figure 1. Codebook-based generalization via discrete kernel regression: Bold arrows indicating observed samples, thin arrows indicating created refinements.

highly imbalanced categoricals. These structural irregularities make the joint distribution difficult to model and consequently hard to sample from. To clarify how current research addresses these challenges, and to motivate our approach, we outline the evolution of tabular generators.

A foundational starting point is the classical three-stage pipeline motivated by Sklar’s theorem (1959): (1) transform each feature into a fixed reference marginal domain; (2) learn a model of joint dependence in that domain; and (3) generate synthetic data by sampling from the learned model and inverting the marginal transforms. For synthetic tabular data, Stages (1) and (2) form the training procedure, while Stage (3) constitutes the sampling phase. The pipeline’s central advantage is *modularity*: marginals and dependence can be handled separately. Because the pipeline originates in copula theory, early tabular generators adopted its canonical form, estimating each marginal distribution, applying the probability integral transform (PIT) to obtain features on $[0, 1]$, and fitting a parametric or vine copula in Stage (2) (Patki et al., 2016; Sun et al., 2019).

Modern models seek to replace the copula in Stage (2) with more expressive neural generators, while retaining the practical and theoretically grounded separation of feature distributions and dependence motivating the three-stage pipeline. This requires viewing Stage (1) abstractly: rather than being tied to PIT mappings, it can be any featurewise separable transform $T(X) = (T_1(X_1), \dots, T_d(X_d)) \in \mathcal{Z}_1 \times \dots \times \mathcal{Z}_d$, whose functional role is (i) to factor out

feature wise marginal effects and (ii) to place the features in a domain compatible with the inductive bias of the Stage (2) generator. With this perspective, Stage (2) becomes flexible: choosing the T_i 's appropriately allows GANs, VAEs, diffusion models, or transformers to serve as the dependence model. CTGAN/TVAE (Xu et al., 2019), TabDDPM (Kotelnikov et al., 2023), and TabMT (Gulati & Roysdon, 2023) follow this pattern, using PIT-based, mode-normalised, or per-feature tokenised transformations for the T_i 's. Across these models, the space produced by the marginal transforms inherits the Cartesian product geometry shown above; an arrangement that enables modern generators but that might not always align perfectly with their inductive biases.

A related approach is TabSyn (Zhang et al., 2024), which preserves the three-stage pipeline but augments Stage (2) with a learned continuous latent representation. Rather than modeling dependence directly on the discrete product space $Z_1 \times \dots \times Z_d$, inputs are first mapped into a VAE latent space, and diffusion is applied there, where smoothness-based inductive biases are more naturally expressed.

Another latent representation paradigm, developed for masked-language transformers, is vector-quantized (VQ) codebooks (Esser et al., 2021; Yu et al., 2023). They convert representations into discrete token sequences, enabling transformer-style dependence modeling over a finite vocabulary. Bringing this paradigm to tabular data is non-trivial: unlike images or videos, where codebooks act as information bottlenecks for high-dimensional signals with strong locality and redundancy, tabular data are heterogeneous, low-dimensional, unordered, and often data-limited. In this regime, a codebook cannot rely on a bottleneck effect: synthetic tabular generation requires a code space larger than the training sample, with additional codes trained to decode to sensible new rows. Naive quantization risks the opposite outcome, collapsing to memorizing the observed rows.

We introduce Q-Tab, which, to the best of our knowledge, is the first model to bring VQ-modeling to the tabular domain in a way that directly targets its core difficulty: generalization in low-dimensional, data-scarce settings. Q-Tab retains standard Stage (1) preprocessing and introduces a Stage (2) representation that jointly encodes all features into a single discrete token sequence suitable for language-model transformers. Categorical tokens are directly one-hot tokenised. Central to our approach is a unified, residual lookup-free quantizer (LFQ) that maps all numerical features jointly into a shared discrete code space with an explicit notion of locality. Our design builds on the principle of residual VQ introduced in (Zeghidour et al., 2021), adapting it to a LFQ formulation. A BERT-style Transformer is then trained to model the resulting token dynamics, and, for Stage (3) synthetic data are generated by sampling from this learned token distribution and decoding back to feature space.

LFQ replaces standard VQGAN-style embedding lookups with a combinatorial index space whose size can far exceed the number of available samples. Classical VQ codebooks are poorly suited to the tabular usecase, since their capacity is tied to an explicit learned embedding table rather than to a structured combinatorial domain. Under a squared reconstruction loss and bit corruption during training, the optimal decoder at each code implements a form of discrete kernel regression: predictions correspond to weighted barycenters of training samples associated with nearby codes. Each LFQ token sequence consists of a primary bit sequence, which fixes a coarse semantic anchor, and a secondary (residual) bit sequence, which encodes fine-grained refinements (Fig. 1). Crucially, corruption is applied only to the secondary bits, inducing a corruption kernel on the full discrete code space with strong locality properties. Training the full LFQ stack realizes a moving kernel regression problem at each index of the code space, driven by evolving encoder assignments. More aggressive corruption expands the set of codes that receive meaningful supervision and improves generalization potential, but also increases the difficulty of the learning problem. We show that the locality induced by residual-only corruption mitigates this tradeoff by restricting supervision to semantically coherent neighbourhoods. Adversarial training of the decoder further constrains these interpolations to remain consistent with the data distribution, ensuring that barycentric predictions stay on the data manifold. Finally we use label smoothening for the residual numerical tokens in the MLM loss to later be able to realise the newly created codes.

Empirically, we show that Q-Tab advances the state of the art on six widely used benchmarks, improving predictive utility on average by 23% and direct label prediction by 20%, while matching the distributional data fidelity of strong diffusion-based generators.

2. Related Work

Beyond the models discussed in the introduction (CTGAN, TVAE, TabDDPM, TabMT and TabSyn), two further methods also fit the three-stage pipeline outlined above. TabNat (Zhang et al., 2025) combines transformer blocks with diffusion-style denoising, and TabDiff (Shi et al., 2025) introduces a unified diffusion framework that jointly models mixed numerical and categorical feature distributions.

A number of approaches fall outside this framework. Examples include STaSy (Kim et al., 2022), CoDi (Lee et al., 2023), and GOGGLE (Liu et al., 2023), TabTreeFormer (Li et al., 2025), which pursue score-based, conditional-generation, graph-based or tree-based estimators rather than separating marginal and joint modelling. These methods, however, have not matched the empirical performance of models developed within above pipeline.

3. Background: Lookup-free Quantization

Vector quantization underlies neural discrete latent-variable models such as VQ-VAE (Van Den Oord et al., 2017), which map continuous encoder outputs to discrete latent representations for generative modeling. VQ-GAN (Esser et al., 2021) builds on this framework by combining vector quantization with adversarially trained decoders to improve fidelity.

Lookup-free quantization removes the explicit codebook and nearest-neighbor assignment used in classical vector quantization, which scale poorly in high-capacity models. In LFQ (Yu et al., 2024), an encoder produces a latent vector $z \in \mathbb{R}^B$ whose components are independently binarized via the sign function, yielding $\hat{z} \in \{\pm 1\}^B$ and implicitly indexing a virtual codebook of size 2^B . Training typically includes two entropy-based regularizers: a per-sample entropy term, which encourages confident quantization decisions for individual samples, and a codebook-usage entropy term, which promotes balanced utilization of the implicit codebook across the dataset.

4. Method: Q-Tab

Q-Tab (Fig. 2) follows the three-stage framework outlined in the introduction, with our main contribution in Stage (2), which we factor into two components: (2.1) tokenizes numerical features jointly using a residual VQ-style codebook data and (2.2) models their joint distribution with an MLM-trained transformer.

4.1. Marginal standardization - Stage (1)

Stage (1) follows the standard process also used in (Shi et al., 2025; Zhang et al., 2025; 2024), where margin-specific idiosyncrasies are removed in the Sklar sense. Because the alignment with the Stage (2) model’s inductive bias is handled by the learned representation around it, Q-Tab’s Stage (1) can be kept minimal and statistically principled. We therefore use empirical CDFs for numerical features and one-hot encodings for categoricals.

Let $\mathcal{D} = \{x_i\}_{i=1}^N$ denote the data training sample, with each row decomposed as $x_i = [x_i^{(\text{num})}, x_i^{(\text{cat})}]$. The numerical feature block lies in $\mathbb{R}^{d_{\text{num}}}$ and categorical feature block in $\prod_{j=1}^{d_{\text{cat}}} \{1, \dots, V_j\}$. Categorical variables are represented by one-hot indicators. Numerical features, continuous or discrete, are mapped column-wise to their quantile scale via the empirical CDF $\hat{F}_j$ (PIT transform), yielding $q_i = \hat{F}(x_i^{(\text{num})}) \in [0, 1]^{d_{\text{num}}}$. For repeated values, the empirical CDF maps tied observations to the same quantile, which can create plateaus for the Stage (2) tokenizer. We add a small uniform tie-breaking jitter before applying $\hat{F}_j$, spreading tied values over nearby quantile levels and reducing collapse of the VQ-based tokenizer on repeated numerical values.

4.2. Joint Tokenization - Stage (2.1)

The first part of Stage (2) defines the joint tokenization. We begin by fixing notation and outlining the tokenization procedure. We then present a general learning-theoretic view on discrete code spaces with built in potential to generalize. Subsequently we apply our newly developed theory in the context of Q-Tab to showcase the benefits of residual LFQ.

4.2.1. TOKENIZATION MECHANICS

We first describe the tokenization of numerical features, which forms the core of the joint tokenizer. Let $u \in [0, 1]^{d_{\text{num}}}$ denote the quantile-standardized numerical features obtained from Stage (1). Before tokenization, we apply the componentwise inverse standard normal CDF, yielding $x = \Phi^{-1}(u) \in \mathbb{R}^{d_{\text{num}}}$. This maps bounded quantile coordinates to an unconstrained Euclidean scale, allowing the MLP encoder-decoder used by the tokenizer to reconstruct numerical features without explicit output constraints or boundary handling at 0 and 1. The transformed vector x then serves as input to a *single encoder that jointly processes all numerical features*:

$$E_\phi : \mathbb{R}^{d_{\text{num}}} \longrightarrow \mathbb{R}^{d_{\text{num}} \times B},$$

with B denoting the bit-count per numerical latent block. We denote the encoder output by $h = E_\phi(x)$ and view it as a matrix in $\mathbb{R}^{d_{\text{num}} \times B}$, with rows $h_j \in \mathbb{R}^B$ used for quantization and token construction. Thus, the tokenizer uses one numerical latent block per numerical feature. This is a simplifying design choice rather than a requirement: in our terminology, *downsampling* would mean using fewer latent blocks than numerical features. Since the present one-block-per-feature design scales linearly in d_{num} , it is not intended for ultra-high-dimensional tables. Appendix B.3.5 provides a preliminary look at downsampled numerical tokenization, while a systematic treatment is left for future work.

Each block is discretized using a two-level *residual LFQ* scheme. The first level applies sign quantization,

$$r_j^{(1)} := h_j, \quad b_j^{(1)} = \text{sign}(r_j^{(1)}) \in \{\pm 1\}^B.$$

A second level quantizes the residual left by the first,

$$r_j^{(2)} := r_j^{(1)} - \alpha_1 b_j^{(1)}, \quad b_j^{(2)} = \text{sign}(r_j^{(2)}) \in \{\pm 1\}^B.$$

The quantized representation is then:

$$\hat{h}_j = \alpha_1 b_j^{(1)} + \alpha_2 b_j^{(2)}. \quad (1)$$

with $0 < \alpha_2 < \alpha_1$. The stack of these representations, $\{\hat{h}_1, \dots, \hat{h}_{d_{\text{num}}}\}$ is then passed to the decoder.

Each binary block $b_j^{(m)}$ is mapped to a token ID via:

$$c_j^{(m)} = \sum_{d=1}^B \frac{(b_j^{(m)})_d + 1}{2} 2^{d-1} + 1. \quad (2)$$

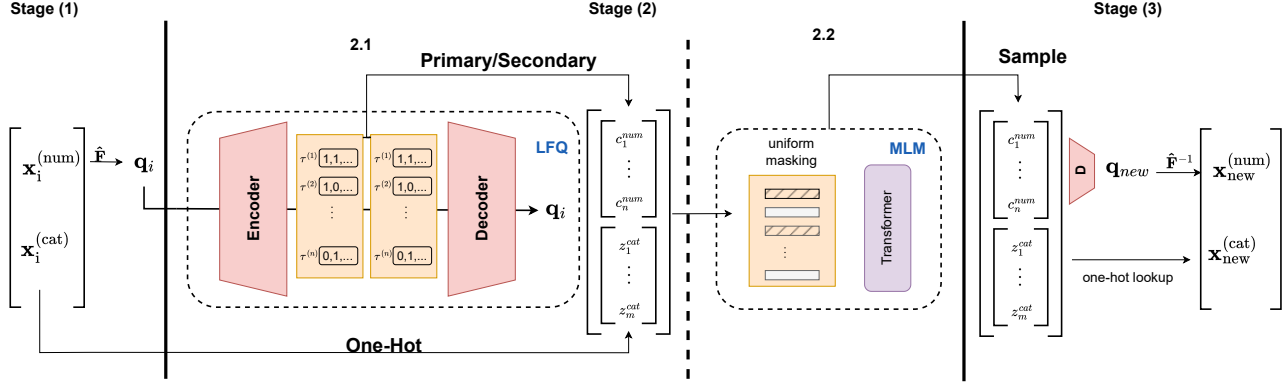


Figure 2. Overview of the Q-Tab pipeline which consists of three stages (details in Section 4). Stage (1): numerical features are mapped to empirical quantiles q_i ; categorical features to one-hot indicators. Stage (2.1): a resnet based encoder, decoder tokenizes the data using residual lookup-free quantization for numerals and appends categorical tokens; Stage (2.2): a BERT-style MLM learns token dependencies. Stage (3): new token sequences are sampled, decoded, and mapped back through the inverted Stage (1) transforms.

yielding a shared numerical vocabulary $\mathcal{V}_{\text{num}} = \{1, \dots, 2^B\}$ for both levels, and we denote the resulting numerical tokens by $c_j^{(m)} \in \mathcal{V}_{\text{num}}$.

During training, the residual bit blocks $b_j^{(2)}$ are independently corrupted by bit flipping, where each bit is flipped with probability $\varepsilon \in (0, 1/2)$. No corruption is applied to the primary blocks. Following LFQ, we retain the per-sample entropy regularizer, omit the codebook-usage entropy term, and add a simple quadratic penalty that penalizes deviations of the latent outputs from zero.

Each data row $\mathbf{x}_i$ is assigned a fixed-length, $L = 2d_{\text{num}} + d_{\text{cat}}$, token sequence $T(\mathbf{x}_i)$ that concatenates the numerical LFQ tokens with the categorical feature tokens,

$$[c_{i,1}^{(1)}, c_{i,1}^{(2)}, \dots, c_{i,d_{\text{num}}}^{(1)}, c_{i,d_{\text{num}}}^{(2)}, z_{i,1}^{(\text{cat})}, \dots, z_{i,d_{\text{cat}}}^{(\text{cat})}], \quad (3)$$

The numerical tokens $c_{i,j}^{(m)}$ from Eq. (2) take values in the shared vocabulary $\mathcal{V}_{\text{num}}$ defined above. We use small bit sizes B , with representational capacity arising from the combinatorial structure across token positions rather than from large per-token vocabularies. Categorical features are then appended as one-hot tokens and assigned consecutive indices starting from $2^B + 1$.

4.2.2. LEARNING A NEURAL CODEBOOK WITH POTENTIAL TO GENERALIZE

We study how a discrete codebook induced by quantization can acquire meaningful decoder values beyond the codes directly assigned to training samples. Let $\{x_i\}_{i=1}^N \subset \mathbb{R}^d$ denote training targets. A continuous encoder latent space is mapped by a deterministic quantization rule to a fixed finite index set $\mathcal{Z}$; at iteration t , this induces assignments $z_{i,t} \in \mathcal{Z}$. A decoder $D_t : \mathcal{Z} \rightarrow \mathbb{R}^d$ defines the codebook $\mathcal{C}_t = \{(z, D_t(z)) : z \in \mathcal{Z}\}$.

For synthetic tabular data, the available index space is deliberately larger than the empirical set of assignments: useful generation requires plausible new points, not only reconstructions of observed samples. We therefore say that such a codebook has *potential to generalize* if training gives sensible decoder values to additional, non-directly-assigned indices. Corruption provides a principled mechanism for doing so: before decoding, an assigned code z_i is replaced by a code $\tilde{z}$ sampled from a designed corruption kernel $q(\cdot | z_i)$, where $q(\tilde{z} | z)$ is the probability of corrupting z into $\tilde{z}$. This exposes additional indices to the training objective, so the full loss stack can act on them, and determines which training targets supervise each such index.

Fixed assignments induce kernel regression. First consider a static encoder, so that the assignments $z_1, \dots, z_N \in \mathcal{Z}$ are fixed. We train an unrestricted decoder $D : \mathcal{Z} \rightarrow \mathbb{R}^d$ under squared reconstruction loss with corrupted codes:

$$\mathcal{L} = \sum_{i=1}^N \mathbb{E}_{\tilde{z} \sim q(\cdot | z_i)} \|D(\tilde{z}) - x_i\|_2^2.$$

The Bayes-optimal squared-loss predictor is a conditional expectation (Vincent et al., 2010; Alain & Bengio, 2014); in the discrete code space, this gives the following kernel-regression form.

Proposition 4.1 (Kernel regression). *For any index $z \in \mathcal{Z}$ that receives nonzero supervision under corruption, the Bayes-optimal decoder satisfies*

$$D^*(z) = \frac{\sum_i q(z | z_i) x_i}{\sum_i q(z | z_i)}.$$

See proof in Appendix A.1. Proposition 4.1 shows that training under corruption propagates supervision beyond the assigned anchors and induces Nadaraya–Watson kernel regression on $\mathcal{Z}$.

Moving assignments yield moving-target regression.

With fixed assignments, Proposition 4.1 gives a fixed kernel-regression target $D^*(z)$. Under joint encoder-decoder training, the assignments $z_{i,t}$ evolve as the encoder is updated, so a fixed index z instead sees a moving kernel-regression target. Fix z and an iteration t with $\sum_{j=1}^N q(z | z_{j,t}) > 0$. We describe the target that supervises z at this iteration by first sampling a training index according to

$$\pi_t(i | z) := \frac{q(z | z_{i,t})}{\sum_{j=1}^N q(z | z_{j,t})}, \quad i = 1, \dots, N.$$

Here $I_t(z) \sim \pi_t(\cdot | z)$ selects one of the training targets whose current assignment can corrupt into z , and $X_t(z) := x_{I_t(z)}$ is the corresponding sampled supervision target for index z . Let $\mathcal{F}_t$ denote the training history up to iteration t and define the instantaneous Bayes target $D_t^*(z) := \mathbb{E}[X_t(z) | \mathcal{F}_t]$, as the optimal current squared-loss predictor. Under squared loss, a standard SGD-style update at index z has, up to scaling, the form $D_{t+1}(z) = (1 - \eta)D_t(z) + \eta X_t(z)$, with tracking error $e_t(z) := D_t(z) - D_t^*(z)$; see Appendix A.2 for a more detailed derivation.

For an unrestricted decoder, this places the local learning problem in the standard stochastic-approximation tracking setting with drifting targets (Kushner & Yin, 2003; Borkar, 2008). Two local quantities govern the learning problem: the *instantaneous supervision variance*,

$$\text{Var}(X_t(z) | \mathcal{F}_t) := \mathbb{E}[\|X_t(z) - D_t^*(z)\|_2^2 | \mathcal{F}_t],$$

which measures dispersion of the targets supervising z , and the *target drift*,

$$\Delta_t(z) := D_{t+1}^*(z) - D_t^*(z),$$

which measures movement of the Bayes target as assignments change.

Proposition 4.2 (Tracking conditions). *Fix an index z and consider its local update process. Assume $\mathbb{E}[\text{Var}(X_t(z) | \mathcal{F}_t)] \leq \sigma^2(z)$ and $\mathbb{E}[\|\Delta_t(z)\|_2^2] \leq v^2(z)$ for all t . Then, for each fixed step size $0 < \eta < 1$, there exists a constant $C_\eta < \infty$, depending on η but independent of z and t , with*

$$\limsup_{t \rightarrow \infty} \mathbb{E}[\|e_t(z)\|_2^2] \leq C_\eta \cdot (\sigma^2(z) + v^2(z)).$$

See proof in Appendix A.2. Proposition 4.2 shows that finite variance and drift bounds are sufficient for bounded asymptotic tracking error, with smaller bounds yielding an easier learning problem. Typically, stochastic approximation uses such bounds to choose the step size η , while the noise and drift are treated as fixed properties of the problem. In Q-Tab, the situation is reversed: η is implicit and not directly tunable, whereas the variance and drift terms can be influenced by the geometry of the corruption kernel q .

To make this geometry precise, we describe which partitions of the code space are preserved by the support of q . Call a finite segmentation $G : \mathcal{Z} \rightarrow \mathcal{G}$ *support-respecting* for q if $q(z | z') > 0$ implies $G(z) = G(z')$. Thus corruption can only move supervision within compartments, never across them. For any G , define $Y_t(z) := G(z_{I_t(z),t})$, the compartment label of the sampled target’s current assignment.

Lemma 4.3 (Compartment decomposition of variance). *When the conditional distribution at time t is well defined,*

$$\begin{aligned} \text{Var}(X_t(z) | \mathcal{F}_t) &= \mathbb{E}[\text{Var}(X_t(z) | Y_t(z), \mathcal{F}_t) | \mathcal{F}_t] \\ &\quad + \text{Var}(\mathbb{E}[X_t(z) | Y_t(z), \mathcal{F}_t] | \mathcal{F}_t). \end{aligned}$$

Lemma 4.4 (Compartment decomposition of drift). *Let $p_t(g | z) := \mathbb{P}(Y_t(z) = g | \mathcal{F}_t)$ and $\mu_t(g | z) := \mathbb{E}[X_t(z) | Y_t(z) = g, \mathcal{F}_t]$. Then, whenever the relevant conditional distributions at t and $t + 1$ are well defined,*

$$\begin{aligned} \Delta_t(z) &= \sum_{g \in \mathcal{G}} p_{t+1}(g | z) (\mu_{t+1}(g | z) - \mu_t(g | z)) \\ &\quad + \sum_{g \in \mathcal{G}} (p_{t+1}(g | z) - p_t(g | z)) \mu_t(g | z). \end{aligned}$$

In both identities, the first term is within-compartment and the second is cross-compartment.

Proposition 4.5 (Support-locality removes cross-compartment channels). *Let G be support-respecting for q , and set $Y_t(z) := G(z_{I_t(z),t})$. If z is updated at time t , then*

$$Y_t(z) = G(z) \quad \mathcal{F}_t\text{-a.s.}$$

Hence the cross-compartment variance term in Lemma 4.3 vanishes; if z is updated at both t and $t + 1$, the cross-compartment drift term in Lemma 4.4 vanishes as well.

See proofs in Appendix A.3. Together with Proposition 4.2, the decompositions yield the schematic tracking structure $a \leq b + c$, where b contains within-component effects and c cross-component mixing. Proposition 4.5 gives $c = 0$ for any support-respecting segmentation.

Among support-respecting segmentations, we use the finest one because it gives the most resolved view of the locality imposed by q . Define the *support-component segmentation* $G_q : \mathcal{Z} \rightarrow \mathcal{G}_q$ by connecting z, z' in the symmetrized support graph whenever $q(z | z') > 0$ or $q(z' | z) > 0$, and letting $G_q(z)$ be the connected component containing z . This lens makes the corruption tradeoff explicit: larger support spreads supervision to more codes, supporting generalization beyond directly assigned indices, but can also mix incompatible assignments. A useful kernel should therefore have support components large enough to share supervision, yet aligned with the encoder-decoder geometry so that the remaining within-component variance and drift can be handled by the model.

4.2.3. BENEFITS OF RESIDUAL LFQ IN Q-TAB

Residual corruption in LFQ realizes this regime by aligning kernel locality with encoder–decoder geometry. The variance and drift decompositions concern the kernel-induced supervision signal and therefore still apply in Q-Tab; what does not transfer directly is the pointwise tracking bound. Proposition 4.2 assumes independent values $D_t(z)$, whereas Q-Tab uses a shared decoder D_{θ_t} whose values $D_{\theta_t}(z)$ are coupled through the MLP. Thus variance and drift remain the quantities to control, but no longer give a per-code tracking theorem for the full neural dynamics.

For d_{num} numerical features, we stack primary and residual blocks as in Eq. (1) and define $\mathcal{Z} = \{\pm\alpha_1\}^{Bd_{\text{num}}} \times \{\pm\alpha_2\}^{Bd_{\text{num}}}$, with $z = (z^{(1)}, z^{(2)})$. The decoder receives the summed LFQ code $z^{(1)} + z^{(2)}$, which lies in the shared encoder-latent/decoder-input space $\mathbb{R}^{Bd_{\text{num}}}$, whose coordinatewise sign patterns define orthants. Since $\alpha_1 > \alpha_2 > 0$, the orthant of $z^{(1)} + z^{(2)}$ is determined by $z^{(1)}$, while $z^{(2)}$ refines the code within that orthant. Thus, for each primary sign pattern $g \in \{\pm\alpha_1\}^{Bd_{\text{num}}}$, we define the corresponding discrete orthant in code space $\mathcal{Z}_g = \{z \in \mathcal{Z} : z^{(1)} = g\}$.

Residual corruption keeps primary blocks fixed and flips residual bits independently. For $u, v \in \{\pm\alpha_2\}^B$, let $q_\varepsilon(u | v) = (1 - \varepsilon)^{B - d_H(u, v)} \varepsilon^{d_H(u, v)}$, where d_H is Hamming distance. With this, the residual LFQ kernel is

$$q(z | z') = \prod_{k=1}^{d_{\text{num}}} \mathbf{1}\{z_k^{(1)} = z_k'^{(1)}\} q_\varepsilon(z_k^{(2)} | z_k'^{(2)}).$$

Hence $q(z | z') > 0$ implies $z^{(1)} = z'^{(1)}$, so corruption cannot cross orthants. Conversely, for $0 < \varepsilon < 1$, every residual pattern inside a fixed orthant has positive corruption probability. Thus the finest support-respecting segmentation G_q is exactly the orthant partition $\{\mathcal{Z}_g\}_g$.

Residual LFQ thus aligns the corruption geometry with the architecture: the support components are exactly the orthants of the shared encoder-latent/decoder-input space. By Proposition 4.5, cross-orthant variance and drift channels vanish from the kernel-induced supervision signal, while supervision still spreads within each orthant. The remaining within-orthant terms must be handled by the parametrized decoder. Orthants are meaningful because codes in one orthant share a coarse sign pattern, making coherent interpolation compatible with the smoothness bias of bounded-weight encoder–decoder MLPs. Learnability also relies on stabilized assignments and within-orthant kernel locality: residual flips weight supervision by residual Hamming distance. Since these properties depend on the learned representation and restricted parametrization, we assess them empirically. Section 6.1 isolates this effect by comparing residual LFQ with a flat LFQ variant whose globally supported kernel has a trivial support-component partition.

We now turn to the remaining components of the LFQ training objective beyond the reconstruction loss. The encoder-latent margin penalties introduced in Section 4.2.1 further stabilize assignments and thereby support the tracking behavior analyzed above. We omit the LFQ codebook-usage entropy term, as it counteracts stable assignments and is unnecessary when expressivity arises from combinations across token positions rather than usage balancing within a single vocabulary. Finally, the decoder is trained with an adversarial loss, which is orthogonal to the reconstruction-loss-induced kernel regression and serves to correct distributional biases introduced by barycentric averaging.

4.3. Learning Token Dynamics – Stage (2.2)

Stage (2.2) operates purely in token space. The dataset is represented as a token matrix $T(\mathcal{D}) \in [V]^{N \times (2d_{\text{num}} + d_{\text{cat}})}$, following Eq. (3), and a BERT-style Transformer is trained with MLM to learn the joint distribution over tokens.

Residual LFQ induces a coarse-to-fine tokenization of the numerical features: primary tokens encode global regimes, while residual tokens encode refinements within a fixed primary sequence. At the token-dynamics modeling stage, two objects are present: the realized token sequence produced by the encoder for each training sample, and the surrounding codespace learned in Stage (2.1). In the neighborhood of each encoded training example, the codespace already supports meaningful local generalization: nearby residual refinements that preserve the same primary tokens have been trained to decode sensibly. From the Transformer’s perspective, however, this structure is only potential, since it is not directly observable from a single realized sequence.

Residual label smoothing is the mechanism by which this latent potential in the codespace is activated under the MLM objective. At a masked position i , the loss is the cross-entropy between the target distribution $p(y | i)$ and the model prediction $q_\theta(y | i)$. Primary numerical and categorical tokens use a Dirac target. For residual numerical tokens, we apply label smoothing around the observed token y_i^* , assigning $p(y_i^* | i) = 1 - \delta$ and distributing the remaining mass uniformly over the residual vocabulary. This enforces a nonzero lower bound on the predicted probability of every residual token, preventing rarely or not at all observed refinements from being suppressed and allowing probability mass to be shaped by contextual compatibility rather than encoder assignment. In this way, residual label smoothing enables the Transformer to exploit the generalization structure already present in the codespace.

Finally, masking follows (Gulati & Roysdon, 2023): for each row, we draw $p \sim \mathcal{U}[0, 1]$ and independently mask each token with probability p . This exposes the model to varied context sizes during training and matches the autoregressive sampling.

4.4. Synthetic Data Generation - Stage (3)

Stage (3) specifies the full path from token sampling back to the original data space. Synthetic rows are generated by sampling a token sequence from the Transformer and deterministically decoding it through the frozen components of Stages (2.1) and (1). Numerical tokens are mapped through the LFQ decoder and inverted using the standard normal CDF followed by the marginal inverses F_j^{-1} , while categorical tokens are decoded by one-hot lookup. At this point, the only set screw is the order in which tokens are generated.

Sampling proceeds autoregressively directly from the model’s predictive distribution, *without* temperature scaling or other post-hoc modifications. Because the Transformer was trained with uniformly random masking, it supports conditional prediction under arbitrary partial contexts (Gulati & Roysdon, 2023). We sample tokens in a fixed coarse-to-fine order: first categorical tokens, then primary numerical tokens, and finally residual numerical tokens. Categorical tokens fix the discrete regime, primary numerical tokens select a coarse numerical region, and residual numerical tokens provide fine-scale refinements. Within each block, tokens are sampled in a random internal order. This order of the blocks matters, as categorical tokens must be sampled in a low-entropy environment: fixing the categorical regime before introducing numerical variation prevents minority modes from being destabilized by noisy numerical contexts.

In the absence of domain knowledge, meaningful novelty cannot arise from inventing unseen categorical combinations or entirely new coarse numerical regions. Instead, novelty arises from new numerical refinements within existing categorical regimes and coarse numerical contexts, realized in Q-Tab through residual LFQ and the sampler above.

5. Experiments

5.1. Datasets, baselines, and training setup

Datasets. Following prior work (Zhang et al., 2024; Shi et al., 2025), we evaluate on six mixed-type UCI benchmarks: Adult, Default, Shoppers, Magic, Beijing, News. 4 Datasets have a binary target value while two have a continuous target value. For more detailed information on the datasets and splits used we refer to Appendix B.1

Baselines. We compare Q-Tab against a broad set of state-of-the-art models: This includes CTGAN (Xu et al., 2019), TVAE (Xu et al., 2019), TabDDPM (Kotelnikov et al., 2023), TabSyn (Zhang et al., 2024), TabDiff (Shi et al., 2025), and TabtNat (Zhang et al., 2025). All implementations were obtained from publicly available codebases.¹

¹<https://github.com/fangliancheng/TabNAT>

Training Details. During training, all models use identical data splits, following the official TabNAT repository.¹ Experiments are conducted on a single RTX 4090 GPU. Q-Tab employs a residual LFQ vector quantizer with a bit size of $B = 4$, paired with a BERT-style Transformer encoder. A detailed overview of the implementation details and all hyperparameters are provided in the Appendix B.2

5.2. Results

For evaluation, we follow established protocols in tabular generative modeling (Shi et al., 2025; Zhang et al., 2025; 2024). We assess downstream utility through Machine Learning Efficiency (MLE) and direct label prediction, and we measure distributional fidelity (DF) using quality metrics from SDMetrics (DataCebo, 2020), which summarizes marginal and joint feature consistency. To ensure that models do not simply replay training records, we report the Distance-to-Closest-Record (DCR) score as defined in (Kotelnikov et al., 2023), which serves as proxy metric for the privacy-preserving properties of the generated data.

Downstream utility (MLE). Table 1 evaluates the standard *train-on-synthetic, test-on-real* protocol across four classification and two regression tasks. Q-Tab attains the *lowest average gap* to real-data trained models at 4.6%, improving over the best prior method TabDiff by 23.3% relative. Per dataset, Q-Tab has the best AUC on Default, Shoppers, and achieves the best RMSE on Beijing. Moreover, the reported variance is the tightest on average and the lowest on all datasets except News, indicating that the observed gains are reproducible rather than artifacts of sampling noise.

Direct label prediction. Following the evaluation protocol of (You et al., 2020), Table 2 evaluates whether generative models can impute masked target columns while preserving downstream predictive performance. For numerical target columns, however, this setting is not naturally aligned with models that use vector quantization to jointly encode numerical features: once several numerical columns are represented by a shared discrete code, the masked target value is not directly exposed as an independent conditional variable. We therefore restrict this analysis to classification targets. Relative to a non-generative XGBoost baseline trained on the same imputed inputs, Q-Tab achieves the largest improvement, exceeding the next best method, TabDiff, by 20.07% in average AUC.

Sample Realism and Coverage We further evaluate sample realism and coverage using α -Precision and β -Recall, which measure fidelity to high-density regions of the real distribution and support coverage, respectively. This is important for Q-Tab because residual LFQ could in principle

Table 1. Reports average Machine-Learning Efficiency (MLE) with a XGBoost (Chen & Guestrin, 2016) model trained on synthetic data (20 seeds), and tested on real data. AUC $\uparrow$ (classification), RMSE $\downarrow$ (regression). Q-Tab outperforms the SOTA by 23.3% (4.6% vs. 6.0%).

Methods	Adult	Default	Shoppers	Magic	Beijing	News	Average Gap
	AUC $\uparrow$	AUC $\uparrow$	AUC $\uparrow$	AUC $\uparrow$	RMSE $\downarrow$	RMSE $\downarrow$	% $\downarrow$
Real	.927 $\pm$.001	.771 $\pm$.003	.925 $\pm$.002	.946 $\pm$.002	.445 $\pm$.007	.834 $\pm$.018	0.0
CTGAN	.881 $\pm$.002	.732 $\pm$.007	.860 $\pm$.018	.882 $\pm$.003	1.076 $\pm$.034	.871 $\pm$.020	28.3
TVAE	.878 $\pm$.002	.736 $\pm$.005	.903 $\pm$.007	.917 $\pm$.003	1.081 $\pm$.023	.954 $\pm$.010	28.8
TabNat	.904 $\pm$.002	.748 $\pm$.007	.912 $\pm$.004	-	.653 $\pm$.013	.873 $\pm$.004	11.7
TabDDPM	.837 $\pm$.012	.762$\pm$.003	.914 $\pm$.004	.934 $\pm$.003	.598 $\pm$.012	-	9.5
TabSyn	.907 $\pm$.001	.754 $\pm$.006	.912 $\pm$.007	.937 $\pm$.002	.568 $\pm$.012	.851$\pm$.024	6.1
TabDiff	.912$\pm$.002	.760 $\pm$.006	.919$\pm$.005	.938$\pm$.003	.568 $\pm$.017	.865 $\pm$.025	6.0
Q-Tab (ours)	.911 $\pm$.001	.762$\pm$.004	.919$\pm$.003	.934 $\pm$.003	.536$\pm$.011	.854 $\pm$.018	4.6

¹ Rows marked - ran out of memory on the corresponding evaluation.

Table 2. Performance of Q-Tab on the direct label prediction task. We compare against the SOTA models TABSYN and TABDIFF; XGBoost serves as a non-generative reference.

Methods	Adult	Default	Shoppers	Magic	Avg. Improv.
	AUC $\uparrow$	AUC $\uparrow$	AUC $\uparrow$	AUC $\uparrow$	% $\uparrow$
Predicted by XGBoost	92.7	77.1	92.5	94.6	0.0
TABSYN	92.7	87.0	96.47	86.1	2.03
TABDIFF ($\omega = 0.6$)	93.2	85.6	96.07	92.6	3.33
Q-Tab (ours)	93.6	86.9	96.1	95.2	4.02

reduce diversity by discretizing numerical variables. Q-Tab achieves the best average α -Precision (0.9863) and β -Recall (0.5150), outperforming the strongest baseline TabDiff on both metrics. These results indicate that Q-Tab preserves high sample fidelity while improving coverage, suggesting that its structured residual code space does not induce mode collapse or reduce diversity. For a detailed overview of the experiment is provided in the Appendix B.3.3.

Table 3. Reports average privacy (DCR; target = 0.5) and average distances to the real-data baseline for DF and MLE across datasets.

Metric	TabSyn	TabDiff	Q-Tab
DF (% $\downarrow$)	2.6	2.0	2.2
Avg. DCR	.513	.542	.529
MLE (% $\downarrow$)	5.5	4.1	3.8

Privacy and distributional fidelity. Table 3 summarizes privacy and fidelity for the three most competitive methods. We measure privacy using DCR, where values closer to 0.5 indicate stronger privacy preservation (Zhang et al., 2024). Since DCR can be improved trivially by generating low-quality samples, we additionally report DF and MLE on the same DCR splits to contextualize privacy results. Overall, TabSyn achieves the best average DCR, but this comes with substantially worse DF and MLE. In contrast, Q-Tab attains the second-best DCR while achieving the best MLE and

competitive DF, yielding the most favorable privacy-utility trade-off. Notably Q-Tab achieved this without temperature tuning. A more comprehensive privacy and quality breakdown is provided in Appendix B.3.

6. Ablation study

6.1. Residual LFQ Setup

Table 4. Comparison of flat and residual LFQ tokenization under bit-flip corruption across datasets. We report duplicate rates, DCR scores, and MLE performance relative to the real-data baseline.

Model	Metric	Shop.	Beij.	Def.	Adult	Mean
LFQ	Dup. (%)	24.87	27.10	3.23	66.95	30.54
	DCR	0.543	0.558	0.533	0.550	0.546
	MLE (%)	1.33	28.06	0.65	1.90	7.99
LFQ +Flip	Dup. (%)	6.62	6.07	0.22	23.50	9.60
	DCR	0.516	0.531	0.509	0.537	0.523
	MLE (%)	2.24	39.30	1.09	2.34	11.24
ResLFQ +Flip	Dup. (%)	6.49	4.09	0.65	19.58	7.70
	DCR	0.525	0.545	0.526	0.538	0.534
	MLE (%)	1.54	27.81	0.23	1.62	7.80

This ablation study examines two claims suggested by the analysis in Section 4.2.2. First, Proposition 4.1 identifies corruption as a mechanism through which generalization can arise in a discrete codebook. Second, while we have shown that residual LFQ eliminates a specific failure channel in the drift and variance terms identified in Proposition 4.2, we hypothesize that the remaining components governing learning dynamics are manageable in practice.

We compare three tokenization schemes: (i) LFQ, a single-stage lookup-free quantizer without corruption; (ii) LFQ+Flip, a ‘flat’ LFQ with uniform bit-flip corruption, inducing a global corruption kernel; and (iii) ResLFQ+Flip, the proposed residual LFQ with bit flipping restricted to

residual blocks. All models share the same downstream architecture, GAN and margin penalties, codebook capacity, and training procedure.

Table 4 reports duplication rates, DCR, and MLE across datasets. Despite strong MLE, LFQ without corruption exhibits substantially higher duplication rates, with DCR showing a corresponding shift, indicating that the learned codebook lacks an inherent mechanism for generalization and empirically supporting Proposition 4.1.

Comparing LFQ+Flip and ResLFQ+Flip isolates the effect of locality. ResLFQ+Flip achieves an even lower duplication rate, reflecting a larger active vocabulary, and consistently attains substantially higher MLE, suggesting more stable learning at the same corruption strength. This supports the second hypothesis: residual structure removes a failure mode associated with global mixing, leaving only within-compartment dynamics that are manageable in practice. In contrast, flat LFQ with global corruption must reconcile supervision drawn from the full code space, leading to high-variance updates and blurred reconstructions. Although this can reduce DCR, it degrades learnability. Residual LFQ mitigates this tradeoff by its locality property, yielding improved MLE and strong generalization.

6.2. Sampler

Table 5. DCR, MLE, and Quality for shoppers and beijing.

Configuration	DCR	AUC $\uparrow$	RMSE $\downarrow$	Quality $\uparrow$
Uniform	0.5801	0.919	0.5318	0.979
Num $\rightarrow$ Cat	0.6176	0.9176	0.5385	0.974
Cat $\rightarrow$ Num	0.534	0.919	0.539	0.982

We now investigate the impact of the proposed ordering structure in Section 4.4. We evaluate privacy (DCR), downstream utility (AUC/RMSE), and statistical similarity. *Uniform* samples tokens in random order, *Num $\rightarrow$ Cat* samples numerical tokens before categorical ones, and *Cat $\rightarrow$ Num* follows the proposed coarse-to-fine hierarchy by anchoring categorical regimes before generating numerical variation.

As shown in Table 5, *Cat $\rightarrow$ Num* consistently achieves the strongest DCR and quality scores. Early categorical anchoring stabilizes discrete regimes and improves numerical conditioning. *Uniform* partially recovers this effect by chance, while *Num $\rightarrow$ Cat* systematically leads to rare category collapse by making categorical decisions last.

MLE-based performance remains effectively unchanged across sampling orders, as the MLE could be insensitive to rare category collapse. Overall, these results confirm that early categorical anchoring is critical for improving distributional fidelity and statistical similarity.

Sensitivity analysis. Tables 18,19,20 in Appendix B.3.6 show that the main hyperparameters trade off utility and diversity. Increasing the LFQ bit size B reduces numeric duplicates by lowering code collisions, but overly large code spaces weaken the per-code training signal; accordingly, $B = 3-4$ gives the best utility, while larger B improves DCR/privacy. For corruption, a small amount is beneficial, with the best AUCROC at $\epsilon = 0.05$, whereas stronger corruption makes supervision too noisy. Moderate MLM label smoothing is also helpful, with weighted-F1 peaking at $\delta = 0.10-0.15$, and reducing numeric duplicates; excessive smoothing again dilutes the learning signal.

7. Conclusion

We introduced Q-Tab as a codebook-based approach to synthetic tabular data, where the central difficulty is not compression but learning discrete representations that generalize beyond observed samples. The results show that vector-quantized latent modeling can support high-quality tabular generation, yielding strong predictive utility and imputation performance while maintaining favorable privacy behavior. Taken together, Q-Tab establishes codebook-based latent modeling as a viable foundation for tabular data.

Limitations. Q-Tab in its current form makes several simplifying design choices. Its tokenization leads to sequence lengths that scale linearly with the number of features, increasing both training and sampling cost; see Appendix B.3.4. Categorical features are represented by direct one-hot tokens, which is simple but may be suboptimal under high cardinality or strong imbalance. The token masking scheme aims at learning all conditional distributions and is not tailored to the sampling scheme. The model also uses a fixed hyperparameter configuration rather than dataset-specific adaptation.

Outlook. These limitations suggest several directions for future work. More compact tokenizations, shorter sequence representations, and learned categorical embeddings could improve scalability and representation quality. Masking could be synced with sampling. Data-dependent hyperparameter choices and adaptive training schemes could further strengthen the analytical framework developed here. A particularly relevant direction is to control the occupancy of support components induced by the corruption kernel: ensuring that active components receive sufficient assignments would make corruption-based supervision propagation more reliable and could reduce sensitivity to the number of orthants. More broadly, we hope that the learning-theoretic perspective introduced in this work will help guide future codebook-based methods beyond the specific design choices and applications of Q-Tab.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Alain, G. and Bengio, Y. What regularized auto-encoders learn from the data-generating distribution. *The Journal of Machine Learning Research*, 15(1):3563–3593, 2014.
- Assefa, S. A., Dervovic, D., Mahfouz, M., Tillman, R. E., Reddy, P., and Veloso, M. Generating synthetic data in finance: opportunities, challenges and pitfalls. In *Proceedings of the First ACM International Conference on AI in Finance*, ICAIF ’20, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450375849. doi: 10.1145/3383455.3422554. URL <https://doi.org/10.1145/3383455.3422554>.
- Becker, B. and Kohavi, R. Adult. UCI Machine Learning Repository, 1996. <https://archive.ics.uci.edu/dataset/2/adult>. Accessed: 2026-01-03.
- Bock, R. Magic gamma telescope. UCI Machine Learning Repository, 2004. <https://archive.ics.uci.edu/dataset/159/magic+gamma+telescope>. Accessed: 2026-01-03.
- Borkar, V. S. *Stochastic approximation: a dynamical systems viewpoint*, volume 48. Springer, 2008.
- Chen, S. Beijing pm2.5 data. UCI Machine Learning Repository, 2014. <https://archive.ics.uci.edu/dataset/381/beijing+pm2+5+data>. Accessed: 2026-01-03.
- Chen, T. and Guestrin, C. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD ’16, pp. 785–794. ACM, August 2016. doi: 10.1145/2939672.2939785. URL <http://dx.doi.org/10.1145/2939672.2939785>.
- DataCebo. Sdmetrics: Synthetic data metrics. <https://github.com/sdv-dev/SDMetrics>, 2020. Version 0.7.0.
- Dua, D. and Graff, C. Uci machine learning repository. <https://archive.ics.uci.edu/>, 2019. Accessed: 2026-01-03.
- Esser, P., Rombach, R., and Ommer, B. Taming transformers for high-resolution image synthesis, 2021. URL <https://arxiv.org/abs/2012.09841>.
- Fernandes, K., Vinagre, P., Cortez, P., and Sernadela, P. Online news popularity. UCI Machine Learning Repository, 2015. <https://archive.ics.uci.edu/dataset/332/online+news+popularity>. Accessed: 2026-01-03.
- Fonseca, J. and Bacao, F. Tabular and latent space synthetic data generation: a literature review. *Journal of Big Data*, 10:1–37, July 2023. ISSN 2196-1115. doi: 10.1186/s40537-023-00792-7. info:eu-repo/grantAgreement/FCT/6817 - DCRRNI ID/UIDB
- Gulati, M. S. and Roysdon, P. F. Tabmt: Generating tabular data with masked transformers, 2023. URL <https://arxiv.org/abs/2312.06089>.
- Hernandez, M., Epelde, G., Alberdi, A., Cilla, R., and Rankin, D. Synthetic data generation for tabular health records: A systematic review. *Neurocomputing*, 493:28–45, 2022. ISSN 0925-2312. doi: <https://doi.org/10.1016/j.neucom.2022.04.053>. URL <https://www.sciencedirect.com/science/article/pii/S0925231222004349>.
- Kim, J., Lee, C., and Park, N. Stasy: Score-based tabular data synthesis. *arXiv preprint arXiv:2210.04018*, 2022.
- Kotelnikov, A., Baranchuk, D., Rubachev, I., and Babenko, A. Tabddpm: Modelling tabular data with diffusion models. In *International conference on machine learning*, pp. 17564–17579. PMLR, 2023.
- Kushner, H. J. and Yin, G. G. *Stochastic approximation and recursive algorithms and applications*. Springer, 2003.
- Lee, C., Kim, J., and Park, N. Codi: Co-evolving contrastive diffusion models for mixed-type tabular synthesis. In *International Conference on Machine Learning*, pp. 18940–18956. PMLR, 2023.
- Li, J., Zhao, B., Zhao, Z., Javaid, U., Yee, K., and Sikdar, B. Tabtreeformer: Tabular data generation using hybrid tree-transformer, 2025. URL <https://arxiv.org/abs/2501.01216>.
- Liu, T., Qian, Z., Berrevoets, J., and van der Schaar, M. Goggle: Generative modelling for tabular data by learning relational structure. In *The Eleventh International Conference on Learning Representations*, 2023.
- Patki, N., Wedge, R., and Veeramachaneni, K. The synthetic data vault. In *2016 IEEE international conference on data science and advanced analytics (DSAA)*, pp. 399–410. IEEE, 2016.
- Sakar, C. O. Online shoppers purchasing intention dataset. UCI Machine Learning Repository, 2018. <https://archive.ics.uci.edu/dataset/>

- 468/online+shoppers+purchasing+intention+dataset. Accessed: 2026-01-03.
- Shi, J., Xu, M., Hua, H., Zhang, H., Ermon, S., and Leskovec, J. Tabdiff: a mixed-type diffusion model for tabular data generation, 2025. URL <https://arxiv.org/abs/2410.20626>.
- Sklar, A. Fonctions de répartition à n dimensions et leurs marges. *Publications de l'Institut de Statistique de l'Université de Paris*, 8:229–231, 1959.
- Sun, Y., Cuesta-Infante, A., and Veeramachaneni, K. Learning vine copula models for synthetic data generation. In *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence and Thirty-First Innovative Applications of Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence*, AAAI'19/IAAI'19/EAAI'19. AAAI Press, 2019. ISBN 978-1-57735-809-1. doi: 10.1609/aaai.v33i01.33015049. URL <https://doi.org/10.1609/aaai.v33i01.33015049>.
- Van Den Oord, A., Vinyals, O., et al. Neural discrete representation learning. *Advances in neural information processing systems*, 30, 2017.
- Vincent, P., Larochelle, H., Lajoie, I., Bengio, Y., Manzagol, P.-A., and Bottou, L. Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion. *Journal of machine learning research*, 11(12), 2010.
- Xu, L., Skoularidou, M., Cuesta-Infante, A., and Veeramachaneni, K. Modeling tabular data using conditional gan. *Advances in neural information processing systems*, 32, 2019.
- Yeh, I.-C. Default of credit card clients. UCI Machine Learning Repository, 2009. <https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients>. Accessed: 2026-01-03.
- You, J., Ma, X., Ding, D. Y., Kochenderfer, M., and Leskovec, J. Handling missing data with graph representation learning, 2020. URL <https://arxiv.org/abs/2010.16418>.
- Yu, L., Cheng, Y., Sohn, K., Lezama, J., Zhang, H., Chang, H., Hauptmann, A. G., Yang, M.-H., Hao, Y., Essa, I., et al. Magvit: Masked generative video transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 10459–10469, 2023.
- Yu, L., Lezama, J., Gundavarapu, N. B., Versari, L., Sohn, K., Minnen, D., Cheng, Y., Birodkar, V., Gupta, A., Gu, X., Hauptmann, A. G., Gong, B., Yang, M.-H., Essa, I., Ross, D. A., and Jiang, L. Language model beats diffusion – tokenizer is key to visual generation, 2024. URL <https://arxiv.org/abs/2310.05737>.
- Zeghidour, N., Luebs, A., Omran, A., Skoglund, J., and Tagliasacchi, M. Soundstream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 30:495–507, 2021.
- Zhang, H., Zhang, J., Srinivasan, B., Shen, Z., Qin, X., Faloutsos, C., Rangwala, H., and Karypis, G. Mixed-type tabular data synthesis with score-based diffusion in latent space, 2024. URL <https://arxiv.org/abs/2310.09656>.
- Zhang, H., Fang, L., Wu, Q., and Yu, P. S. Tabnat: A continuous-discrete joint generative framework for tabular data. In *Forty-second International Conference on Machine Learning*, 2025.

A. Setup and Proofs for Codebook Learning Results (Section 4.2.2)

This Section provides detailed setups and proofs of the propositions and lemmas stated in Section 4.2.2. Throughout, we work in general, unrestricted encoder-decoder settings, keeping the exposition exact and self-contained. All notation matches the main paper unless stated otherwise.

A.1. Fixed Assignments: Kernel Regression Induced by Corruption

Setup. Let $\mathcal{Z}$ be a finite index set and let $\{(x_i, z_i)\}_{i=1}^N$ be fixed training pairs with targets $x_i \in \mathbb{R}^d$ and fixed assignments $z_i \in \mathcal{Z}$. Let $q : \mathcal{Z} \times \mathcal{Z} \rightarrow [0, 1]$ be a corruption kernel, meaning that

$$\sum_{z \in \mathcal{Z}} q(z | z') = 1 \quad \text{for every } z' \in \mathcal{Z},$$

where $q(z | z')$ denotes the probability that index z' is corrupted into z .

We consider an unrestricted decoder $D : \mathcal{Z} \rightarrow \mathbb{R}^d$ trained with corruption under squared reconstruction loss

$$\mathcal{L}(D) = \sum_{i=1}^N \mathbb{E}_{\tilde{z} \sim q(\cdot | z_i)} \|D(\tilde{z}) - x_i\|_2^2.$$

Proposition A.1 (Kernel regression under corruption). *For each index $z \in \mathcal{Z}^* := \{z \in \mathcal{Z} : \exists i : q(z | z_i) > 0\}$, the Bayes-optimal decoder minimizing $\mathcal{L}(D)$ satisfies*

$$D^*(z) = \frac{\sum_{i=1}^N q(z | z_i) x_i}{\sum_{i=1}^N q(z | z_i)}.$$

Proof. Expanding the objective yields

$$\mathcal{L}(D) = \sum_{i=1}^N \sum_{z \in \mathcal{Z}} q(z | z_i) \|D(z) - x_i\|_2^2 = \sum_{z \in \mathcal{Z}} \sum_{i=1}^N q(z | z_i) \|D(z) - x_i\|_2^2.$$

Since D is unrestricted, the values $\{D(z)\}_{z \in \mathcal{Z}}$ are independent decision variables. For a fixed $z \in \mathcal{Z}^*$, define

$$F_z(y) = \sum_{i=1}^N q(z | z_i) \|y - x_i\|_2^2, \quad y \in \mathbb{R}^d.$$

Because $\sum_i q(z | z_i) > 0$, F_z is a strictly convex quadratic in y , with unique minimizer given by

$$y^* = \frac{\sum_i q(z | z_i) x_i}{\sum_i q(z | z_i)}.$$

Setting $D^*(z) = y^*$ for all $z \in \mathcal{Z}^*$ minimizes $\mathcal{L}(D)$. □

A.2. Learning under Moving Assignments

Setup and tracking formulation. Let $\{x_i\}_{i=1}^N \subset \mathbb{R}^d$ denote fixed training targets. At iteration t , the encoder induces assignments $z_{i,t} \in \mathcal{Z}$. Let $q : \mathcal{Z} \times \mathcal{Z} \rightarrow [0, 1]$ be a fixed corruption kernel as above. Define the effective code set

$$\mathcal{Z}_\infty := \{z \in \mathcal{Z} : \exists j \text{ such that } q(z | z_{j,t}) > 0 \text{ for infinitely many } t\}.$$

Let $(\mathcal{F}_t)_{t \geq 0}$ denote the filtration generated by the training history up to iteration t . Fix $z \in \mathcal{Z}_\infty$. For any iteration t for which there exists at least one j with $q(z | z_{j,t}) > 0$, define

$$\pi_t(i | z) = \frac{q(z | z_{i,t})}{\sum_{j=1}^N q(z | z_{j,t})}.$$

Conditionally on $\mathcal{F}_t$, let

$$I_t(z) \sim \pi_t(\cdot | z), \quad X_t(z) := x_{I_t(z)} \in \mathbb{R}^d.$$

The instantaneous Bayes target is then

$$D_t^*(z) := \mathbb{E}[X_t(z) | \mathcal{F}_t].$$

Consider a decoder value $D_t(z) \in \mathbb{R}^d$ updated whenever there exists at least one $j \in \{1, \dots, N\}$ with $q(z | z_{j,t}) > 0$, by

$$D_{t+1}(z) = (1 - \eta) D_t(z) + \eta X_t(z), \quad 0 < \eta < 1.$$

Define the tracking error, noise, and drift terms

$$e_t(z) := D_t(z) - D_t^*(z), \quad \xi_t(z) := X_t(z) - D_t^*(z), \quad \Delta_t(z) := D_{t+1}^*(z) - D_t^*(z).$$

A direct calculation yields the recursion

$$e_{t+1}(z) = (1 - \eta) e_t(z) + \eta \xi_t(z) - \Delta_t(z),$$

with

$$\mathbb{E}[\xi_t(z) | \mathcal{F}_t] = 0, \quad \mathbb{E}[\|\xi_t(z)\|_2^2 | \mathcal{F}_t] = \text{Var}(X_t(z) | \mathcal{F}_t),$$

where for a vector-valued random variable Y , we use the scalar conditional variance

$$\text{Var}(Y | \mathcal{F}) := \mathbb{E}[\|Y - \mathbb{E}[Y | \mathcal{F}]\|_2^2 | \mathcal{F}].$$

Equivalently, this is the trace of the conditional covariance matrix.

Proposition A.2 (Tracking conditions). *Fix $z \in \mathcal{Z}_\infty$ and assume that there exists at least one j with $q(z | z_{j,t}) > 0$ for all t . Assume that there exist constants $\sigma^2(z)$ and $v^2(z)$ such that*

$$\mathbb{E}[\text{Var}(X_t(z) | \mathcal{F}_t)] \leq \sigma^2(z), \quad \mathbb{E}[\|\Delta_t(z)\|_2^2] \leq v^2(z) \quad \text{for all } t.$$

Then, for each fixed step size $0 < \eta < 1$, there exists a constant $C_\eta < \infty$, depending on η but independent of z and t , such that

$$\limsup_{t \rightarrow \infty} \mathbb{E}[\|e_t(z)\|_2^2] \leq C_\eta (\sigma^2(z) + v^2(z)).$$

Proof. Abbreviate $e_t := e_t(z)$, $\xi_t := \xi_t(z)$, $\Delta_t := \Delta_t(z)$. From the recursion,

$$e_{t+1} = (1 - \eta)e_t + \eta\xi_t - \Delta_t.$$

Taking squared norms and expectations yields

$$\begin{aligned} \mathbb{E}\|e_{t+1}\|_2^2 &= (1 - \eta)^2 \mathbb{E}\|e_t\|_2^2 + \eta^2 \mathbb{E}\|\xi_t\|_2^2 + \mathbb{E}\|\Delta_t\|_2^2 \\ &\quad + 2(1 - \eta)\eta \mathbb{E}\langle e_t, \xi_t \rangle - 2(1 - \eta)\mathbb{E}\langle e_t, \Delta_t \rangle - 2\eta \mathbb{E}\langle \xi_t, \Delta_t \rangle. \end{aligned}$$

Since e_t is $\mathcal{F}_t$ -measurable and $\mathbb{E}[\xi_t | \mathcal{F}_t] = 0$, we have $\mathbb{E}\langle e_t, \xi_t \rangle = 0$. The remaining cross terms have uncontrolled sign, so we bound them in absolute value. By Cauchy–Schwarz and Young’s inequality $2ab \leq \alpha a^2 + \alpha^{-1}b^2$, we have

$$2(1 - \eta)|\langle e_t, \Delta_t \rangle| \leq 2\|e_t\|_2((1 - \eta)\|\Delta_t\|_2) \leq \eta(1 - \eta)\|e_t\|_2^2 + \frac{1 - \eta}{\eta}\|\Delta_t\|_2^2,$$

where the last step uses $\alpha = \eta(1 - \eta)$, and, with $\alpha = 1$:

$$2\eta|\langle \xi_t, \Delta_t \rangle| \leq 2\eta\|\xi_t\|_2\|\Delta_t\|_2 \leq \eta^2\|\xi_t\|_2^2 + \|\Delta_t\|_2^2.$$

Substituting these bounds into the squared-error expansion gives

$$\mathbb{E}\|e_{t+1}\|_2^2 \leq (1 - \eta)\mathbb{E}\|e_t\|_2^2 + 2\eta^2\mathbb{E}\|\xi_t\|_2^2 + \frac{2}{\eta}\mathbb{E}\|\Delta_t\|_2^2.$$

Using

$$\mathbb{E}\|\xi_t\|_2^2 = \mathbb{E}[\text{Var}(X_t(z) \mid \mathcal{F}_t)] \leq \sigma^2(z) \quad \text{and} \quad \mathbb{E}\|\Delta_t\|_2^2 \leq v^2(z),$$

the one-step bound becomes

$$\mathbb{E}\|e_{t+1}\|_2^2 \leq (1 - \eta)\mathbb{E}\|e_t\|_2^2 + 2\eta^2\sigma^2(z) + \frac{2}{\eta}v^2(z).$$

Iterating this recursion gives

$$\mathbb{E}\|e_t\|_2^2 \leq (1 - \eta)^t \mathbb{E}\|e_0\|_2^2 + \left(2\eta^2\sigma^2(z) + \frac{2}{\eta}v^2(z)\right) \sum_{k=0}^{t-1} (1 - \eta)^k.$$

Since

$$\sum_{k=0}^{t-1} (1 - \eta)^k \leq \frac{1}{\eta},$$

we obtain

$$\limsup_{t \rightarrow \infty} \mathbb{E}\|e_t\|_2^2 \leq 2\eta\sigma^2(z) + \frac{2}{\eta^2}v^2(z).$$

Equivalently, for each fixed step size $0 < \eta < 1$, there exists a constant $C_\eta < \infty$, depending on η but independent of z and t , such that

$$\limsup_{t \rightarrow \infty} \mathbb{E}\|e_t(z)\|_2^2 \leq C_\eta(\sigma^2(z) + v^2(z)).$$

For example, one may take $C_\eta = 2\eta^{-2}$. □

A.3. Compartment Decompositions of Variance and Drift and Effects of Support Locality

We work with an arbitrary finite segmentation $G : \mathcal{Z} \rightarrow \mathcal{G}$ and define the induced partition label

$$Y_t(z) := G(z_{I_t(z), t}).$$

Let $p_t(g \mid z) := \mathbb{P}(Y_t(z) = g \mid \mathcal{F}_t)$ and $\mu_t(g \mid z) := \mathbb{E}[X_t(z) \mid Y_t(z) = g, \mathcal{F}_t]$, with the convention that $\mu_t(g \mid z) = 0$, whenever $p_t(g \mid z) = 0$.

Lemma A.3 (Compartment decomposition of variance). *Fix $z \in \mathcal{Z}$ and t such that $\sum_{j=1}^N q(z \mid z_{j,t}) > 0$. Then*

$$\begin{aligned} \text{Var}(X_t(z) \mid \mathcal{F}_t) &= \mathbb{E}[\text{Var}(X_t(z) \mid Y_t(z), \mathcal{F}_t) \mid \mathcal{F}_t] \\ &\quad + \text{Var}(\mathbb{E}[X_t(z) \mid Y_t(z), \mathcal{F}_t] \mid \mathcal{F}_t). \end{aligned}$$

Proof. This is the law of total variance applied to

$$\text{Var}(X_t(z) \mid \mathcal{F}_t) = \mathbb{E}[\|X_t(z) - \mathbb{E}[X_t(z) \mid \mathcal{F}_t]\|_2^2 \mid \mathcal{F}_t]$$

with conditioning variable $Y_t(z)$. □

Lemma A.4 (Compartment decomposition of drift). *Fix $z \in \mathcal{Z}$ and t such that $\sum_{j=1}^N q(z \mid z_{j,t}) > 0$ and $\sum_{j=1}^N q(z \mid z_{j,t+1}) > 0$. Then the one-step drift $\Delta_t(z) := D_{t+1}^*(z) - D_t^*(z)$ admits the decomposition*

$$\Delta_t(z) = \sum_{g \in \mathcal{G}} p_{t+1}(g \mid z) (\mu_{t+1}(g \mid z) - \mu_t(g \mid z)) + \sum_{g \in \mathcal{G}} (p_{t+1}(g \mid z) - p_t(g \mid z)) \mu_t(g \mid z).$$

Proof. By the law of total expectation,

$$D_t^*(z) = \mathbb{E}[X_t(z) \mid \mathcal{F}_t] = \sum_{g \in \mathcal{G}} p_t(g \mid z) \mu_t(g \mid z),$$

and an analogous expression holds for $D_{t+1}^*(z)$. Subtracting yields

$$\Delta_t(z) = \sum_{g \in \mathcal{G}} (p_{t+1}(g | z) \mu_{t+1}(g | z) - p_t(g | z) \mu_t(g | z)).$$

Adding and subtracting $p_{t+1}(g | z) \mu_t(g | z)$ inside the sum gives

$$\Delta_t(z) = \sum_{g \in \mathcal{G}} p_{t+1}(g | z) (\mu_{t+1}(g | z) - \mu_t(g | z)) + \sum_{g \in \mathcal{G}} (p_{t+1}(g | z) - p_t(g | z)) \mu_t(g | z),$$

□

Proposition A.5 (Support-locality removes cross-compartment channels). *Let $G : \mathcal{Z} \rightarrow \mathcal{G}$ be support-respecting for q , meaning that*

$$q(z | z') > 0 \implies G(z) = G(z').$$

Define

$$Y_t(z) := G(z_{I_t(z), t}).$$

Then, whenever z is updated at time t ,

$$Y_t(z) = G(z) \quad \mathcal{F}_t\text{-a.s.}$$

Consequently, $Y_t(z)$ is $\mathcal{F}_t$ -almost surely constant, so the cross-compartment variance term in Lemma A.3 vanishes. If z is updated at both t and $t + 1$, the cross-compartment drift term in Lemma A.4 also vanishes.

Proof. Fix z and an update time t . If $I_t(z) = i$ has positive conditional probability given $\mathcal{F}_t$, then by definition of $\pi_t(\cdot | z)$,

$$q(z | z_{i,t}) > 0.$$

Since G is support-respecting,

$$G(z_{i,t}) = G(z).$$

Therefore

$$Y_t(z) = G(z_{I_t(z), t}) = G(z) \quad \mathcal{F}_t\text{-a.s.}$$

Since $Y_t(z)$ is $\mathcal{F}_t$ -almost surely constant,

$$\mathbb{E}[X_t(z) | Y_t(z), \mathcal{F}_t] = \mathbb{E}[X_t(z) | \mathcal{F}_t] \quad \mathcal{F}_t\text{-a.s.}$$

and hence

$$\text{Var}(\mathbb{E}[X_t(z) | Y_t(z), \mathcal{F}_t] | \mathcal{F}_t) = 0.$$

This is the cross-compartment term in Lemma A.3.

If z is updated at both t and $t + 1$, the same argument gives

$$p_t(g | z) = p_{t+1}(g | z) = \mathbf{1}\{g = G(z)\}$$

for every $g \in \mathcal{G}$. Therefore

$$p_{t+1}(g | z) - p_t(g | z) = 0$$

for all g , and the second term in Lemma A.4 vanishes. □

B. Experiments

B.1. Datasets

All datasets were obtained from the UCI Machine Learning Repository (Dua & Graff, 2019). For each dataset we report the number of samples and the number of numerical and categorical *feature* columns (the prediction target is excluded from these counts). In particular we used the same preprocessing code as ³ and the same splits.

Table 6. Overview of the datasets used in our analysis.

Dataset	Task	# Rows	# Num	# Cat	Train	Test
Adult (Becker & Kohavi, 1996)	Classification	48842	6	8	32561	16281
Magic (Bock, 2004)	Classification	19019	10	0	17117	1902
Default (Yeh, 2009)	Classification	30000	14	9	27000	3000
Shoppers (Sakar, 2018)	Classification	12330	10	7	11097	1233
News (Fernandes et al., 2015)	Regression	39644	45	2	35679	3965
Beijing (Chen, 2014)	Regression	41757	6	5	37581	4176

DCR splits. For the distance-to-closest-record (DCR) evaluation, we use the same test and training size. In particular we follow the splits and implementation from ³.

Table 7. Split sizes for the DCR dataset variants.

Dataset (DCR)	Train	Test	Total
Adult (DCR split)	24421	24421	48842
Beijing (DCR split)	20878	20879	41757
News (DCR split)	19822	19822	39644
Default (DCR split)	15000	15000	30000
Shoppers (DCR split)	6165	6165	12330

B.2. Implementation Details for Stage (2)

B.2.1. STAGE (2.1): RESIDUAL LFQ ENCODER–DECODER

We model the numerical block with a residual MLP autoencoder and residual lookup-free quantization. The LFQ implementation was inspired by ⁴. For $D = d_{\text{num}}$ numerical features, the encoder jointly maps $\mathbf{x} \in \mathbb{R}^D$ to latents $\mathbf{h} \in \mathbb{R}^{D \times B}$, and a mirrored residual MLP decoder reconstructs $\hat{\mathbf{x}} \in \mathbb{R}^D$ from the quantized latents. Each MLP layer uses LayerNorm and SiLU activations with the following projections (256, 512, 512, 256). Quantization then uses a residual stack of Q sign quantizers with straight-through gradients, using the scale parameters $\alpha_1 = 1$ and $\alpha_2 = 0.5$; during training, only residual bits are corrupted via independent bit flips with probability ε (primary bits uncorrupted).

The generator is trained with a weighted reconstruction, vector quantize and adversarial objective, where the vector-quantize term includes a commitment penalty and entropy regularization over the implicit binary code space. Adversarial sharpening is provided by a 2-layer MLP with a width of 256, LeakyReLU and dropout. The adversarial loss term consists of BCE-with-logits, mild label smoothing (target 0.9).

B.2.2. STAGE (2.2): MASKED LANGUAGE MODEL OVER JOINT TOKENS

After Stage 2.1, each table row is serialized into a fixed-length token sequence $\mathbf{t} \in \{0, \dots, V-1\}^T$ by concatenating all primary and residual numerical codebook tokens followed by categorical tokens, where V is the joint vocabulary size and T is the context length. We train a masked-token objective by sampling a per-example masking rate $p_m \sim \text{Uniform}(0, 1)$ and independently masking each position with probability p_m , replacing masked tokens with a dedicated [MASK] token and computing loss only on masked positions.

The model is a BERT-style Transformer encoder with learned token and positional embeddings and a stack of L encoder

³<https://github.com/MinkaiXu/TabDiff>

⁴<https://github.com/lucidrains/vector-quantize-pytorch>

layers with H attention heads, GELU activations, dropout, and LayerNorm. Predictions are produced with a *column-aware* head: categorical positions use separate classifiers per column, while numerical codebook positions share a single classifier; we additionally enforce position-specific supports by masking logits for invalid token IDs to $-\infty$ using a precomputed validity mask. We use selective label smoothing whose strength differs across token types/positions (e.g., applied to residual numerical-codebook tokens but not to primary numerical or categorical tokens). Optimization uses AdamW with weight decay, gradient clipping, a learning-rate schedule with linear warmup followed by cosine decay.

B.2.3. HYPERPARAMETERS

We use the hyperparameters summarized in Table 8, covering device settings, model architecture, optimization, and codebook configuration. These values match the configuration used in all Q-Tab experiments, except for bit size, codebook depth, and VQ weight, which were adjusted for the single and residual codebook case. Most hyperparameters were fixed a priori to reasonable default values. Only a small subset was tuned on the smallest dataset (Shoppers) using MLE performance as the target objective. For LFQ, we tuned the learning rate, bit dropout probability, adversarial weight, ratio weight, and reconstruction weight, while for the MLM stage we tuned the learning rate and label smoothing. All selected hyperparameters were then held constant across datasets and experiments.

Table 8. Residual LFQ and MLM hyperparameters used in our experiments.

Residual LFQ Hyperparameters		MLM Hyperparameters	
Category	Value	Category	Value
Batch size	256	Batch size	512
Epochs	2000	Epochs	2000
Learning rate	0.0003	Learning rate	0.0007
Bit size	8,4	Transformer size	128
Codebook depth	1,2	Layers	8
Beta	0.25	Heads	2
Commitment cost	0.25	Label smoothing	0.1
VQ weight (α)	[1.0],[1.0,0.5]	Codebook size	8,4
Bit dropout p	0.07	Codebook depths	1,2
Entropy loss weight	0.5		
Entropy temperature	0.1		
Entropy gamma	0.0		
Adversarial weight	0.9		
Ratio weight	0.1		
Reconstruction weight	6.0		

B.3. Results

B.3.1. DCR

Table 9 reports Distance-to-Closest-Record (DCR) scores across five datasets. Following prior work (Kotelnikov et al., 2023), values closer to 0.5 indicate stronger privacy preservation. We exclude the *Magic* dataset, as it is not provided in the official TabDiff repository.⁵

A favorable DCR can be obtained trivially if a model fails to capture the dependence structure of the underlying data distribution (e.g., by sampling with high post-hoc temperature, Q-Tab uses no temperature scaling). To control for this effect, Table 10 reports both distributional quality and downstream predictive utility (AUROC for classification, RMSE for regression) for models trained under the DCR splits. Overall, the three methods achieve comparable predictive performance while maintaining high quality scores, suggesting that the reported DCR trends are not explained by degraded sample fidelity.

B.3.2. DATA QUALITY

Tables 11, 12, and 13 report the SDMetrics (DataCebo, 2020) quality scores for *Shape*, *Trend*, and the combined *Quality*, respectively.

⁵<https://github.com/MinkaiXu/TabDiff>

Table 9. Distance-to-Closest-Record (DCR) score (Kotelnikov et al., 2023) to measure privacy preservation. The ideal target is 0.5.

Dataset	TabSyn	TabDiff	Q-Tab
Adult	.512	.555	.539
Default	.516	.557	.527
Shoppers	.524	.524	.525
News	.509	.541	.511
Beijing	.505	.535	.545

Table 10. Quality score (density quality score) and predictive utility (AUROC for classification; RMSE for regression). We further provide the avg. distance to the real baselines.

Dataset	TabSyn		TabDiff		Q-Tab	
	Quality $\uparrow$	AUROC/RMSE	Quality $\uparrow$	AUROC/RMSE	Quality $\uparrow$	AUROC/RMSE
Default	0.974	0.764	0.980	0.762	0.970	0.763
Adult	0.983	0.910	0.986	0.912	0.983	0.910
Shoppers	0.976	0.918	0.979	0.919	0.978	0.919
Beijing	0.951	0.704	0.981	0.630	0.979	0.618
News	0.981	0.868	0.972	0.874	0.982	0.872
Avg. distance	0.026	0.055	0.020	0.041	0.022	0.038

B.3.3. ADDITIONAL FIDELITY AND COVERAGE ANALYSIS

In addition to the downstream MLE and distributional metrics in the main paper, we assess sample realism and coverage using α -Precision and β -Recall. α -Precision measures whether generated samples fall in high-density regions of the real data distribution, reflecting fidelity, while β -Recall measures coverage of the real data support, reflecting diversity.

This analysis is especially relevant for Q-Tab, since residual LFQ could in principle reduce diversity by discretizing numerical variables into a structured code space. The results show the opposite: Q-Tab preserves high fidelity while achieving the best average coverage among compared methods.

Q-Tab obtains the highest average α -Precision, see Table 14, scoring 0.9863 versus 0.9859 for TabDiff, the strongest baseline. Although the margin is small, it indicates that residual quantization does not compromise sample fidelity. Q-Tab also achieves the best α -Precision on Adult, Default, and Beijing, and remains competitive elsewhere.

The β -Recall results, see Table 15, show clearer gains in coverage. Q-Tab achieves the best average score, 0.5150, compared with 0.5077 for TabDiff and 0.4800 for TabSyn. It performs best on Shoppers, Magic, Beijing, and News, and remains close to TabDiff on Adult and Default. These results suggest that Q-Tab does not concentrate generation around a narrow subset of the data manifold, but instead preserves broad support coverage and improves the fidelity–coverage trade-off.

Table 11. Statistical fidelity (SDMetrics Trend). Synthetic-trained, real-test. Score $\uparrow$. Q-TAB achieves competitive fidelity relative to SOTA diffusion baselines.

Methods	Adult	Default	Shoppers	Magic	Beijing	News	Average
	Quality $\uparrow$	Quality $\uparrow$	Quality $\uparrow$	Quality $\uparrow$	Quality $\uparrow$	Quality $\uparrow$	Quality $\uparrow$
CTGAN	.738 $\pm$.009	.691 $\pm$.008	.757 $\pm$.002	.943 $\pm$.004	.786 $\pm$.001	.949 $\pm$.000	.811
TVAE	.687 $\pm$.007	.799 $\pm$.027	.761 $\pm$.003	.955 $\pm$.006	.695 $\pm$.024	.941 $\pm$.001	.806
TabNat	.893 $\pm$.001	.886 $\pm$.013	.919 $\pm$.001	-	.925 $\pm$.002	.943 $\pm$.002	.913
TabDDPM	.421 $\pm$.001	.959 $\pm$.001	.942 $\pm$.002	.988 $\pm$.003	.957 $\pm$.003	-	.853
TabSyn	.955 $\pm$.003	.922 $\pm$.012	.974 $\pm$.001	.974 $\pm$.003	.971 $\pm$.002	.982 $\pm$.000	.963
TabDiff	.983$\pm$.003	.963$\pm$.012	.981 $\pm$.001	.989$\pm$.002	.974$\pm$.001	.984$\pm$.001	.979
Q-TAB (ours)	.977 $\pm$.005	.943 $\pm$.029	.982$\pm$.001	.986 $\pm$.004	.973 $\pm$.003	.983 $\pm$.002	.974

¹ Rows marked *OOM* ran out of memory on the corresponding evaluation.

Table 12. Statistical fidelity (SDMetrics Shape). Synthetic-trained, real-test. Score $\uparrow$. Q-TAB achieves SOTA together with TabDiff

Methods	Adult	Default	Shoppers	Magic	Beijing	News	Average
	Quality $\uparrow$	Quality $\uparrow$	Quality $\uparrow$	Quality $\uparrow$	Quality $\uparrow$	Quality $\uparrow$	Quality $\uparrow$
CTGAN	.820 $\pm$.001	.844 $\pm$.001	.730 $\pm$.002	.945 $\pm$.002	.804 $\pm$.001	.858 $\pm$.000	.834
TVAE	.765 $\pm$.001	.915 $\pm$.001	.732 $\pm$.001	.962 $\pm$.001	.726 $\pm$.001	.814 $\pm$.000	.819
TabDDPM	.543 $\pm$.001	.978 $\pm$.001	.972 $\pm$.001	.990 $\pm$.001	.989 $\pm$.001	-	.894
TabNat	.929 $\pm$.001	.905 $\pm$.001	.933 $\pm$.002	-	.930 $\pm$.001	.891 $\pm$.000	.918
TabSyn	.980 $\pm$.001	.949 $\pm$.001	.984 $\pm$.001	.988 $\pm$.001	.988 $\pm$.001	.979 $\pm$.000	.978
TabDiff	.994$\pm$.001	.987$\pm$.001	.986$\pm$.001	.992$\pm$.001	.990$\pm$.000	.977 $\pm$.000	.988
Q-TAB (ours)	.992 $\pm$.000	.987$\pm$.001	.986$\pm$.001	.991 $\pm$.001	.990$\pm$.001	.980$\pm$.000	.988

¹ Rows marked *OOM* ran out of memory on the corresponding evaluation.

Table 13. Statistical fidelity (SDMetrics Quality). Synthetic-trained, real-test. Score $\uparrow$. Q-TAB achieves competitive fidelity relative to SOTA diffusion baselines.

Methods	Adult	Default	Shoppers	Magic	Beijing	News	Average
	Quality $\uparrow$	Quality $\uparrow$	Quality $\uparrow$	Quality $\uparrow$	Quality $\uparrow$	Quality $\uparrow$	Quality $\uparrow$
CTGAN	.779 $\pm$.005	.768 $\pm$.004	.743 $\pm$.002	.944 $\pm$.002	.795 $\pm$.001	.904 $\pm$.000	.822
TVAE	.726 $\pm$.004	.857 $\pm$.013	.747 $\pm$.002	.958 $\pm$.003	.710 $\pm$.012	.877 $\pm$.001	.813
TabDDPM	.482 $\pm$.001	.968 $\pm$.001	.957 $\pm$.001	.989 $\pm$.002	.973 $\pm$.001	-	.874
TabNat	.911 $\pm$.001	.895 $\pm$.006	.926 $\pm$.001	-	.928 $\pm$.001	.917 $\pm$.001	.916
TabSyn	.967 $\pm$.001	.936 $\pm$.006	.979 $\pm$.001	.981 $\pm$.001	.980 $\pm$.001	.981 $\pm$.000	.971
TabDiff	.988$\pm$.002	.975$\pm$.006	.984$\pm$.001	.991$\pm$.001	.982$\pm$.001	.980 $\pm$.001	.983
Q-Tab (ours)	.985 $\pm$.003	.965 $\pm$.014	.984$\pm$.001	.989 $\pm$.002	.982$\pm$.002	.982$\pm$.001	.981

¹ Rows marked *OOM* ran out of memory on the corresponding evaluation.

Overall, α -Precision and β -Recall further support the main findings: Q-Tab combines realistic samples with strong distributional coverage, indicating that its structured residual code space and locality-inducing corruption do not cause mode collapse or reduced diversity.

Table 14. α -Precision of synthetic data generated by each method. Higher values indicate better fidelity to high-density regions of the real data distribution. Score $\uparrow$. Q-TAB achieves the best average α -Precision.

Methods	Adult	Default	Shoppers	Magic	Beijing	News	Average
	α -Prec. $\uparrow$	α -Prec. $\uparrow$	α -Prec. $\uparrow$	α -Prec. $\uparrow$	α -Prec. $\uparrow$	α -Prec. $\uparrow$	α -Prec. $\uparrow$
CTGAN	.7781	.6805	.7079	.8617	.9521	.9742	.8258
TVAE	.6740	.9288	.4857	.9616	.7955	.9133	.7932
TabNat	.9765	.8463	.9503	-	.9619	.8114	.9093
TabSyn	.7308	.9520	.9913	.9946	.9733	.9435	.9309
TabDiff	.9883	.9865	.9905	.9956	.9827	.9720	.9859
Q-Tab (ours)	.9887	.9907	.9880	.9917	.9888	.9697	.9863

B.3.4. RUNTIME ANALYSIS

We complement the main experiments with a runtime comparison. All experiments were run on a single NVIDIA RTX 4090 GPU with 24GB of memory, using the same protocol as the main evaluation. We report total training and synthetic-data generation time for each method. For approaches with separate preprocessing or representation-learning stages, we report the combined training cost; for TabSyn, this includes both VAE pretraining and diffusion-model training.

Q-Tab is not the fastest method overall, see Table 16, but it occupies a practical middle ground among recent high-performing

Table 15. β -Recall of synthetic data generated by each method. Higher values indicate better coverage of the real data distribution. Score $\uparrow$. Q-TAB achieves the best average β -Recall.

Methods	Adult	Default	Shoppers	Magic	Beijing	News	Average
	β -Recall $\uparrow$	β -Recall $\uparrow$	β -Recall $\uparrow$	β -Recall $\uparrow$	β -Recall $\uparrow$	β -Recall $\uparrow$	β -Recall $\uparrow$
CTGAN	.1635	.1312	.2492	.1581	.3679	.2273	.2162
TVAE	.1399	.2848	.1772	.3806	.0600	.2926	.2225
TabNat	.3956	.2955	.4619	-	.5272	.3758	.4112
TabSyn	.4732	.4113	.4935	.4813	.5777	.4430	.4800
TabDiff	.5221	.5191	.5031	.4787	.5929	.4300	.5077
Q-Tab (ours)	.4941	.4953	.5048	.5166	.6065	.4725	.5150

tabular generators. Its training time is lower than TabDiff’s, requiring 2925 time units versus 3575, an approximately 18% reduction. At inference, Q-Tab is substantially faster than TabDDPM and TabNat, requiring 3.8 time units compared with 11.2 and 5.72, respectively. However, it remains slower than lighter baselines such as CTGAN and TVAE.

This comparison illustrates Q-Tab’s main computational trade-off. Residual LFQ tokenization and Transformer-based token modeling add cost relative to simpler baselines, but this cost remains moderate compared with several recent high-performing models and is accompanied by the utility, coverage, and privacy–utility gains observed in the main experiments. A key overhead comes from tokenization: each numerical feature yields primary and residual tokens, making sequence length scale linearly with feature count. This works for low- and medium-dimensional datasets but may be costly for high-dimensional tables, motivating more compact tokenizations, such as using fewer latent blocks than numerical features, to improve Q-Tab’s efficiency.

Table 16. Runtime comparison of Q-Tab and baseline tabular generators. Lower values indicate faster runtime. Training denotes total model fitting time, while inference denotes the time required to generate synthetic samples under the main experimental protocol.

Method	Training $\downarrow$	Inference $\downarrow$
CTGAN	792	0.72
TVAE	400	0.432
TabNat	1958	5.72
TabDDPM	812	11.2
TabSyn	1159 + 456	1.14
TabDiff	3575	2.60
Q-Tab	2925	3.8

B.3.5. COMPACT TOKENIZATION VIA LATENT DOWNSAMPLING

Q-Tab normally uses one latent block per numerical feature, producing one primary and one residual token per feature. This avoids an additional design choice but makes the numerical sequence length scale linearly with d_{num} .

We test a compact variant that reduces the number of numerical latent blocks by 25% while keeping the remaining protocol fixed. Table 17 shows that this shorter tokenization preserves most utility: MLE gaps and DCR scores remain broadly comparable to standard Q-Tab, with small dataset-dependent changes. The main cost is reduced sample uniqueness, as duplicate rates increase by about 2.8 percentage points on average. This is expected, since fewer latent positions shrink the effective discrete code space. Overall, moderate latent downsampling offers an efficiency–uniqueness trade-off: it shortens training and sampling sequences while largely preserving predictive utility and relative privacy behavior. Data-adaptive compact tokenizations are therefore a promising direction for scaling Q-Tab to high-dimensional tables.

B.3.6. SENSITIVITY TO CODE SIZE, CORRUPTION, AND SMOOTHING.

Table 18 shows the effect of the LFQ bit size B . Increasing B expands the discrete code space, which reduces collision pressure and lowers the number of repeated synthetic numeric rows. For example, numeric duplicates decrease from 584 at $B = 3$ to 350 at $B = 5$ and remain below the $B = 3$ level at $B = 6$. At the same time, larger code spaces distribute the fixed

Table 17. Effect of compact numerical tokenization obtained by reducing the number of numerical latent blocks by 25%. Lower duplicate rate and lower MLE gap are better. DCR is reported as the relative nearest-neighbor score used in the main paper.

Dataset	Downsampled Q-Tab			Standard Q-Tab		
	Duplicate rate ↓	DCR	MLE gap ↓	Duplicate rate ↓	DCR	MLE gap ↓
Shoppers	8.58%	.540	1.03%	6.49%	.535	1.54%
Default	1.02%	.520	0.66%	0.65%	.526	0.23%
Adult	27.08%	.535	1.78%	19.85%	.538	1.62%
Beijing	5.39%	.546	27.38%	4.09%	.545	27.81%

training data over more possible codes, reducing the effective training signal per code. This explains why performance does not improve monotonically with B : $B = 3-4$ gives the strongest MLE AUROC, while larger values improve privacy/DCR but slightly reduce utility. Table 19 reports the corruption probability ϵ . Increasing ϵ does not monotonically improve performance. Small-to-moderate corruption improves generalization by encouraging local interaction between nearby codes, with the best AUROC at $\epsilon = 0.05$. However, larger values make the supervision increasingly diffuse: codes are trained against noisier mixtures, which weakens learnability and reduces downstream utility. Finally, Table 20 shows the effect of MLM label smoothing δ . Moderate smoothing gives the best utility, with weighted-F1 peaking around $\delta = 0.10-0.15$. Smoothing also consistently reduces numeric duplicates, from 802 at $\delta = 0$ to 393 at $\delta = 0.20$, indicating that softer targets discourage over-concentration on repeated reconstructions. Very large smoothing, however, starts to dilute the learning signal and reduces MLE performance.

Table 18. Sensitivity to LFQ bit size B on Shoppers. MLE is reported as AUROC; duplicates are synthetic numeric duplicate rows.

B	MLE AUROC ↑	Density quality ↑	DCR ↓	Duplicates ↓
3	0.9208 ± 0.0028	0.9767 ± 0.0012	0.5330	584
4	0.9198 ± 0.0031	0.9780 ± 0.0018	0.5398	374
5	0.9132 ± 0.0037	0.9831 ± 0.0016	0.5251	350
6	0.9152 ± 0.0054	0.9772 ± 0.0015	0.5186	388

Table 19. Sensitivity to corruption probability ϵ on Shoppers.

ϵ	MLE AUROC ↑	Density quality ↑
0.00	0.9163 ± 0.0056	0.9799 ± 0.0010
0.01	0.9181 ± 0.0062	0.9836 ± 0.0008
0.05	0.9215 ± 0.0030	0.9773 ± 0.0017
0.10	0.9184 ± 0.0036	0.9782 ± 0.0021
0.20	0.9177 ± 0.0037	0.9848 ± 0.0008
0.30	0.9163 ± 0.0060	0.9811 ± 0.0012
0.40	0.9177 ± 0.0033	0.9808 ± 0.0008

Table 20. Sensitivity to label smoothing δ on Shoppers.

δ	MLE AUROC ↑	Weighted-F1 ↑	DCR ↓	Duplicates ↓
0.000	0.9183 ± 0.0029	0.6525 ± 0.0142	0.5251	802
0.025	0.9170 ± 0.0036	0.6591 ± 0.0169	0.5267	725
0.050	0.9176 ± 0.0044	0.6541 ± 0.0170	0.5335	606
0.100	0.9189 ± 0.0060	0.6670 ± 0.0148	0.5251	487
0.150	0.9176 ± 0.0030	0.6681 ± 0.0189	0.5239	399
0.200	0.9154 ± 0.0039	0.6441 ± 0.0232	0.5189	393